Testing Policy

Version: 1.2
Last Updated: 04 September 2026

momently

Prepared by

CYBERNAUTS x momentum

momently
Timesheets AI

(Used fonts and colours from the brand style kit)

Introduction

Purpose

The purpose of this Testing Policy is to establish rules and approach to software testing throughout the Momently project. The policy set out the objectives, responsibilities, standards, tools, environments and procedures to verify the system is stable.

This policy applies to the entirety of Momently. It applies to new features, refactoring, security updates, new versions and deployment changes.

Standards

The project follows these standards:

1. Test early and continuously
2. Every Pull Request must pass automated tests
3. Errors should be handled before merging
4. Regression testing is required after major changes
5. Production deployment only use successful pipeline builds

Testing Types

Unit Testing

Purpose

To verify that classes, functions and components or modules are isolated and tested for correctness. For all the frontend, backend and ai services related files need to be tested for correctness. There may be use of mock data for external dependencies.

The **pattern 'AAA'** will be followed.

ARRANGE - During this phase, an isolated component of the application that requires testing is initialized

ACT - During this phase, probe the 'SUT' with targeted test conditions.
SUT is the system under test. It is the specific application or hardware component that is being evaluated during a testing activity.

ASSERT - The result behaviour of the SUT is verified. The actual outcome must match the expected behaviour.

Integration Testing

Purpose

To verify that there is communication between various components. All modules, APIs and databases work together as a group. The strategy to be used is **Sandwich Approach** to ensure communication across all modules, APIs and databases.

The Approach to be followed:

Sandwich (Bottom Up + Top Down

Backend would be the Bottom-Up Approach.
- Testing from the database layer upwards to the API controllers.

Frontend would be Top-Down Approach.
- Testing from the UI downwards, mocking external network boundaries.

Regression Testing

Purpose

This testing must be executed to ensure that new code changes do not break existing functionality or introduce unexpected side effects.

This strategy need to followed and implemented in such cases:
1. Bug Fixes
2. Feature Enhancements
3. Dependency Upgrades
4. Docker Images Updates

End to End (E2E) Testing

Purpose

To test the software application's flow from start to finish. This involves stimulating a real user's scenarios to validate that all subsystems are working together. Make use of the user stories of the current sprint to help to simulate this real user's scenarios.

Tools and Environments

Development

Purpose

All developers are testing before committing. All developers need to ensure that they validate code changes before pushing them to the repository. This will help to catch errors and bugs earliest.

Momently will use Docker Compose to orchestrate the system locally, ensure that you have Docker and Docker Compose, use DBeaver to use as GUI to easily navigate PostgreSQL.

CI Environment

Purpose

This workflow is created to automate the process of such as building, testing, security scanning, pushing built images to AWS ECR and more. This helps developers build, test and deploy software consistently. The approach helps with faster development cycles, improved code quality and reduced manual intervention.

Production

Purpose

The Production environment hosts the final, deployed application on AWS EC2. This is what the real users interact with. The setup is containerized and orchestrated directly on the virtual server instances.

The system makes use of deploy.yml workflow that orchestrates the entire deployment alongside with the CI/CD pipelines. The deploy.yml must always pass and will be triggered on push to the main branch.

Testing Tools

Activity	Tool
Frontend Unit Tests	Jasmine
Backend Unit Tests	JUnit 5
Backend Mocking	Mockito
AI Testing	Pytest
API Testing	Postman / Swagger UI
Code Quality	SonarQube
Container Testing	Docker
CI/CD	GitHub Actions

Acceptance Criteria

A feature is accepted only if:

- Functional requirements are satisfied
- Unit tests pass
- Integration tests pass
- CI/CD pipeline completes successfully
- Docker containers deploy successfully
- The feature is part of the current sprint and approving review through a Pull Request.

Roles of Stakeholders

Developers implement the features, write unit tests, perform local testing and fix defects.